INTELIGENCIA ARTIFICIAL - PRÁCTICA 1 - MEMORIA

Autores: Ana Olsson Rodríguez y Martín Darío Marmolejo Díaz, grupo 09

Índice

1. Respuesta a las preguntas del apartado A..... 2
Pregunta 1.1: ¿Cómo evoluciona el número de nodos al aumentar la profundidad?..... 2
Pregunta 1.2: ¿Es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida? Justifica tu respuesta..... 2
Pregunta 1.3: ¿Qué diferencias observáis entre BP y BA al explorar árboles de juego?..... 2
Pregunta 1.4: ¿Cuál de los dos algoritmos requiere más memoria? Explica por qué.....3
Pregunta 1.5: Suponed que cada hoja contiene una puntuación fija (por ejemplo, en función del número de fichas negras). ¿Permiten estos algoritmos determinar cuál es la mejor jugada para un jugador racional? Justificad vuestra respuesta.....3
Pregunta 1.6: Define 3 funciones heurísticas con varios niveles de complejidad (puedes empezar a partir de la heurística entregada simpleHeuristic), implementando las funciones ya definidas heuristic1, heuristic2, heuristic3 (de menor a mayor complejidad): ¿qué características parecen ayudar a resolver el problema de capturar una esquina en un menor número de pasos?..... 3
Pregunta 1.7: Analiza qué ocurre con los métodos de búsqueda y las distintas heurísticas propuestas si se intenta resolver el problema de capturar todas las esquinas, es decir, en el main, instancia el problema AllCornersReversiSearchProblem en lugar de CornerReversiSearchProblem..... 4
Pregunta 1.8: En base a tus resultados, ¿puede la función heurística equivocarse? ¿qué aspectos habría que tener en cuenta para hacer que A* sea óptimo?.....4
2. Documentación de minmax y poda alfa-beta..... 4
a. Detalles de la implementación.....4
3. Documentación del diseño de la heurística..... 5
a. Trabajos previos en heurísticas en Reversi..... 5
b. Proceso de diseño..... 5
c. Heurísticas finales.....6
Bibliografía..... 7

1. Respuesta a las preguntas del apartado A.

Pregunta 1.1: *¿Cómo evoluciona el número de nodos al aumentar la profundidad?*

El número de nodos total equivale al número de nodos de la profundidad anterior sumado al número de hojas de la profundidad actual, ésta siendo calculada sumando todas las posibles jugadas (sucesores) de los nodos de la profundidad anterior. Si hacemos la media de sucesores por nivel, el aumento es aproximadamente equivalente al número de hojas de la profundidad previa multiplicado por esta media.

En general, si consideramos b el número de jugadas disponibles promedio y d la profundidad del árbol (número de jugadas de la partida que efectivamente se juegan) el número de nodos es de O(b^d), equivalente al coste de explorar un árbol de juego con minmax y correspondiente a una evolución exponencial según la profundidad.

Pregunta 1.2: *¿Es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida? Justifica tu respuesta.*

Depende de la altura y anchura del tablero, ya que ésta determina la profundidad máxima (número de jugadas hasta acabar una partida). En el caso del tablero 6x6 que se utiliza en la función *main* de *search.py*, la profundidad será de aproximadamente 32 por partida (cuando se rellena el tablero).

Tras haber realizado ejecuciones con *iterations_initial_state* con distintos valores se estima que el promedio de jugadas es alrededor de 5 (no se ha calculado experimentalmente), por lo que el coste podría ser aproximadamente 5^32, que equivale a 2.3283064e+22. Concluimos que no es viable el cálculo de árbol de juego para tableros de dimensión superior o igual a ésta, definiendo viable como asequible en un tiempo razonable.

Pregunta 1.3: *¿Qué diferencias observáis entre BP y BA al explorar árboles de juego?*

Varias: primero, BP no es óptimo, sino que una vez encuentra un estado meta dentro de la rama que esté explorando, devuelva ésta como solución, mientras que con BA, al recorrer nivel por nivel todo el árbol, encontrará primero el estado solución que se encuentre a menor profundidad (óptimo). Añadido a esto, dado que BP recorre una única rama del árbol de juego hasta encontrar un estado meta o quedarse sin posibles nodos que explorar, tenderá a explorar una menor cantidad de nodos antes de acabar, contrastando con BA, que tendrá que recorrer todos los nodos de cada nivel que explore antes de encontrar la solución, por lo que los nodos visitados tenderán a ser mayores en BA y de esta manera también el tiempo de ejecución. A continuación se muestran ejemplos de ejecución en los que se observan las características antes descritas:

iterations_initial_state = 0

```
DFS found a path of 9 moves visiting 9 nodes in 0:00:00.002656: [('c', 2), ('b', 2), ('d', 5), ('c', 5), ('b', 5), ('e', 5), ('b', 3), ('b', 6), ('f', 6)]
BFS found a path of 4 moves visiting 79 nodes in 0:00:00.020853: [('e', 4), ('e', 5), ('e', 6), ('f', 6)]
```

iterations_initial_state = 2

```
DFS found a path of 6 moves visiting 6 nodes in 0:00:00.002199: [('e', 2), ('b', 2), ('d', 1), ('c', 5), ('b', 1), ('a', 1)]
BFS found a path of 3 moves visiting 138 nodes in 0:00:00.059952: [('e', 2), ('f', 2), ('f', 1)]
```

iterations_initial_state = 3

```
DFS found a path of 4 moves visiting 4 nodes in 0:00:00.001249: [('c', 5), ('c', 6), ('b', 5), ('a', 6)]
BFS found a path of 2 moves visiting 8 nodes in 0:00:00.002784: [('b', 4), ('f', 6)]
```

Pregunta 1.4: *¿Cuál de los dos algoritmos requiere más memoria? Explica por qué.*

Dado que BA guarda una cantidad mayor de media de nodos visitados en la lista de nodos visitados, y que la estructura de datos (queue) que usa acaba con más elementos que BP (stack), podemos concluir que BA requiere de más memoria.

Pregunta 1.5: *Suponed que cada hoja contiene una puntuación fija (por ejemplo, en función del número de fichas negras). ¿Permiten estos algoritmos determinar cuál es la mejor jugada para un jugador racional? Justificad vuestra respuesta*

Con su implementación actual no toman en cuenta ningún valor adicional, meramente realizan una comprobación llamando a la función *isGoal()* para comprobar si han encontrado el estado meta, que en el caso de *CornerReversiSearchProblem* solo comprueba si una de las piezas se encuentra en cualquier esquina del tablero del juego *Reversi.*

De esta manera, DFS y BFS son incapaces de determinar la mejor jugada en base a esa puntuación fija adicional.

Pregunta 1.6: *Define 3 funciones heurísticas con varios niveles de complejidad (puedes empezar a partir de la heurística entregada simpleHeuristic), implementando las funciones ya definidas heuristic1, heuristic2, heuristic3 (de menor a mayor complejidad): ¿qué características parecen ayudar a resolver el problema de capturar una esquina en un menor número de pasos?*

La primera heurística, de manera similar a *simpleHeuristicI,* maximiza el número de fichas del jugador actual en el tablero mediante un conteo. La segunda heurística prioriza los estados con mayor número de fichas cercanas a las esquinas mediante un mapa de calor. Por último, la tercera prioriza el estado con mayor número de fichas del jugador actual cerca del centro y casillas libres o del contrincante en el exterior.

Pregunta 1.7: *Analiza qué ocurre con los métodos de búsqueda y las distintas heurísticas propuestas si se intenta resolver el problema de capturar todas las esquinas, es decir, en el main, instancia el problema AllCornersReversiSearchProblem en lugar de CornerReversiSearchProblem.*

BP devuelve una solución al problema que es evidentemente más costosa en cuanto a número de nodos y tiempo (debido al estado meta siendo 4 esquinas en vez de 1) pero que es consistente con el comportamiento descrito en anteriores respuestas.

Por otro lado, BA no termina de ejecutarse debido al gran coste en número de nodos que conlleva en contraste a BP (como fue descrito antes, BA recorre cada nivel de profundidad exhaustivamente).

Pregunta 1.8: *En base a tus resultados, ¿puede la función heurística equivocarse? ¿qué aspectos habría que tener en cuenta para hacer que A* sea óptimo?*

A nivel teórico es sabido que para que A* sea óptimo requiere de heurísticas admisibles, es decir, heurísticas que nunca hacen una sobreestimación de los valores de coste reales para un nodo, sólo estimación certeras o infravaloraciones.

A nivel práctico podemos comprobar que este hecho se cumple vía análisis de cómo responden de manera distinta las heurísticas que inicialmente fueron diseñadas para *CornerReversiSearchProblem* a *AllCornersReversiSearchProblem*, pudiendo observar que las soluciones dadas ya no son óptimas (algunas son distintas a lo que devuelve BA, que sigue siendo óptima).

2. Documentación de minmax y poda alfa-beta.

a. Detalles de la implementación.

A la hora de implementar la poda alfa-beta se ha reutilizado el código de la clase ya implementada de *MiniMaxStrategy*, adecuándola al pseudocódigo especificado en el enunciado, es decir, reestableciendo el valor de beta en min() y alpha en max() cada vez que se encuentre un candidato menor o mayor respectivamente y además comparando por cada llamada realizada a *Min()* o *Max()* el valor de alpha o beta respectivamente con el valor candidato de *minmax*, si este candidato es menor o mayor respectivamente, se realizará la "poda" a través de la devolución del candidato actual seleccionado (así, se detiene la

ejecución de la función en la que se encuentra). Exceptuando esto, todo el resto del código es equivalente a la clase de *MiniMaxStrategy* antes mencionada.

Para comprobar el correcto funcionamiento de alpha-beta pruning, se ha comparado la ejecución de demo-tournament.py usando MinimaxStrategy y MinimaxAlphaBetaStrategy en tournament.py (que es el fichero que usa para ejecutar los torneos demo-tournament.py). Comprobamos que los resultados son iguales, en este caso con nuestras heuristicas morning-victory y underdog:

Con AlphaBetaStrategy:

```
Results for tournament where each game is repeated 100=50x2 times, alternating colors for each player
    total:  David_morning-victory   Goliat_underdog
David_morning-victory   50: --- 50
Goliat_underdog 50: 50  ---

Process finished with exit code 0
```

Con MiniMaxStrategy:

```
Successfully initialised board from array
Results for tournament where each game is repeated 100=50x2 times, alternating colors for each player
    total:  David_morning-victory   Goliat_underdog
David_morning-victory   50: --- 50
Goliat_underdog 50: 50  ---

Process finished with exit code 0
```

3. Documentación del diseño de la heurística.

a. Trabajos previos en heurísticas en Reversi.

Para nuestras heurísticas decidimos leer varios artículos sobre las estrategias empleadas anteriormente para el Reversi y así poder basarnos en ellas para el diseño de nuestras estrategias.

b. Proceso de diseño.

Decidimos fragmentar el diseño aprovechando las tres partes de tal manera que la primera heurística se adaptase al momento del juego actual (early game o late game) ya que había ciertas estrategias que sólo era óptimas al principio, como es el minimizar el número de fichas conquistadas o priorizar la movilidad.

Así pues, *Solution1* es una versión que se adapta en función del número de fichas actual empleando *Solution2*, que se enfoca en capturas posiciones valiosas a través de una representación del tablero 8x8 y teniendo en cuenta si las esquinas han sido capturadas, y

Solution3 que prioriza, de cara al inicio del juego, el mayor número de movimientos posibles y mayor número de fichas rivales cercanas a dichos movimientos (de cara a una captura posterior de estas). Todas incluyen el código sugerido de que si el estado es final se priorice ganar con la mayor diferencia.

c. Heurísticas finales.

Nuestros resultados son una primera solución que pondera varios aspectos en función a la etapa del juego actual, una segunda heurística centrada en tomar ciertas posiciones y una tercera que opta por la mayor movilidad posible.

d. Comentarios adicionales.

Cabe añadir que las heurísticas probablemente puedan ser mejoradas alterando sus ponderaciones ya que variaba mucho su desempeño contra *random* cambiando los porcentajes de peso (sobre todo en la primera solución). Además, creemos que hay situaciones en las que las heurísticas destacan mucho pero otras en la que fallan ya que, tras múltiples iteraciones de *demo_tournament.py* observamos cierta volatilidad en cuanto a resultados, variando el número de victorias en 100 enfrentamientos entre 64-82.

Bibliografía

- Festa, J., & Davino, S. (2013). “IAgo vs Othello”: An artificial intelligence agent playing reversi. *PAI@AI*IA*, 43–50. https://ceur-ws.org/Vol-1107/paper2.pdf
- *The world of computer reversi*. (s/f). The World of Computer Reversi. Recuperado el 22 de octubre de 2026, de https://reversiworld.wordpress.com/category/weighted-square-value/
- (S/f). Datacomm.ch. Recuperado el 22 de octubre de 2026, de http://home.datacomm.ch/t_wolf/tw/misc/reversi/html/index.html#Prio